Tabish Shaikh

Computer Engineering

Company: PhonePe

Job Profile: SDE

Offer Type: Internship + Placement

Internship Stipend: 80k

CTC: 33.5L (19.25L base + 5L joining bonus + 8.5L stocks)

Location: Banglore

Process Date:

- PPT + OA : 7th July, 2025
- Interview : 8th July, 2025

Placement Process:

1. **[Online Assessment]** [219 candidates had applied]

4 Questions were to be solved in 1 hour 30 mins

a. You are given n people, labeled from 1 to n. There are m existing friendships represented by two arrays A and B, each of size m, where the i-th friendship is between person A[i] and person B[i].

   Two people are considered friends if there is a direct or indirect connection between them (i.e., if they belong to the same connected group of friends).

   Your task is to determine the **minimum number of new friendships** that must be formed so that **every person is connected directly or indirectly**—in other words, so that everyone belongs to a single connected group.

   Input: N = 4, M = 2, A = [1, 3], B = [2, 4]
   Output: 1

   This is essentially a **connected components** problem, where each group of friends forms a component, and the **minimum number of new friendships required** is simply:

   Answer = Number of connected components − 1

   This is because each new friendship can merge two components, and to connect k components into one, you need exactly k−1 connections.

b. You have N employees and are given an array P of size N, where P[i] indicates the number of ladoos the i-th employee receives. If P[i] = 0, it means the ladoos for that employee are yet to be assigned. You are also given two integers M and K:

- M is the **maximum** number of ladoos any employee can get.
- K is the **maximum allowed difference** in ladoos between **adjacent employees**.

Your task is to determine the **number of valid ways** to assign ladoos to the unassigned positions such that:

- Each unassigned position gets a value between 1 and M.
- The absolute difference between the ladoos of adjacent employees is at most K.

Input: N = 5, M = 5, K = 2 , P = [0, 3, 0, 0, 4]
Output: 80

Initially, I approached this problem using **backtracking**. For each position with value 0, I tried assigning all values from 1 to M and recursively explored the possibilities while maintaining the K difference constraint between adjacent employees. I maintained a counter to track the number of valid configurations.

This approach worked for a few test cases and passed 4 of them. However, it was **too slow** for larger inputs due to **recomputing overlapping subproblems**, and I quickly realized that this was a **Dynamic Programming** problem.

Although I identified that memoization was required, I **struggled to define the exact DP state due to time constraints**.

Dp State:
Let dp[i][prev] represent the number of ways to fill the array from index i onward given that the previous value was prev. Try all possible values from 1 to M at unassigned positions and memoize results to avoid recomputation.

c. You are given an integer n, an integer k, and an array arr of size n consisting of integers. For each element arr[i], convert it to its binary representation and define its value as:

Sum of (bit × position) where position is **1-based from right** in the binary representation.

You need to find the number of **pairs (i, j)** such that:

value(arr[i]) + value(arr[j]) == k

Input: n = 4, k = 20, arr = [5, 6, 7, 8]
Output: 1

- I looped from the least significant bit to the most significant bit, checking if each bit was set using (num >> bitPos) & 1.

- While doing this, I maintained a position counter (1-based from MSB) and added bit * position to the score.

After calculating the custom value, I stored it in an array and this transformed the question to check if the array has a pair for which the value is equal to k, I solved this using hashmap storing the frequencies and incrementing the count of the final answer.

This approach passed all regular test cases. However, one test case had a **missing array input** despite providing n.
To handle this edge case, I wrapped the input parsing in a **try-catch block**, and in such a case, I returned 0.

Thanks to the optimized bitwise handling and proper input validation, I was able to **solve all test cases correctly**.

d. You are given:
   - Four houses from Harry Potter: **Gryffindor, Hufflepuff, Ravenclaw, and Slytherin**
   - Two integers:
     n: number of students
     m: maximum number of students allowed in each house
   - A list of friendships: pairs of integers (a, b) where student a and student b are friends and must be placed in **different houses**
   - A strings, where the i-th string contains the **allowed houses** for the i-th student (e.g., "[["pref1, pref2"], [pref2]]" you had to parse the string and split them accordingly and store their preferences)

   Your task is to determine if it is **possible** to assign every student to exactly one of their preferred houses such that:

   1. **No pair of friends** is placed in the **same house**
   2. **No house has more than m students**
   3. Each student is assigned to **one of their preferred houses**

   Print "possible" if such an assignment exists, otherwise "impossible".

   This was the **final and most complex question**, especially due to the **tricky input format**:

   - I had to read n and m
   - Then an **unknown number of friend pairs**
   - Followed by a single line of strings indicating **preferences** for each student

   Because I was short on time and the input parsing itself was time-consuming, I chose not to attempt this question.

2. **[Technical Round 1]** [30 candidates were shortlisted]

In this round, we were divided into three panels of 10 students each, with each session lasting approximately an hour. I was a part of the third panel. Some candidates were asked to introduce themselves, while others were immediately given DSA problems. The interview followed a similar pattern to the previous years, although the difficulty level had noticeably increased this time, with topics like **Trie** and **Binary Lifting** also making an appearance.

I was first asked to introduce myself, after which I was presented with a DSA problem:

Problem:
Given an array of integers (which may include both positive and negative numbers), find the **maximum sum** of a subarray that starts and ends with the same value. If no such subarray exists, return the Integer.MIN_VALUE.

I initially explained the brute force approach — iterating through every possible subarray and checking if the first and last elements matched. Then, I discussed an optimized solution using a **hashmap** to store indices of elements and computing prefix sums to calculate the subarray sum in constant time. However, the interviewer pointed out that in cases where all elements are the same, this method could degrade to brute force in terms of time complexity.

To solve the problem efficiently, I used a prefix sum array to compute the sum between any two indices in constant time and maintained a hash map to track the first occurrence of each number. While iterating through the array, if a number had been seen before, I calculated the sum of the subarray between its first occurrence and the current index. If this sum was greater than the current maximum, I updated the answer. In cases where the computed sum was negative, I reset the first occurrence of that number to the current index to allow for potentially better subarrays in the future. For numbers appearing for the first time, I simply stored their index in the map.

This logic struck a good balance between efficiency and edge case handling. The interviewer seemed satisfied with the clarity and completeness of my explanation and asked me to write down the code for it.

The second question I was asked involved a binary tree and multiple queries, where each query was a pair (node, value)—and the same node could appear multiple times with different values. For each query, the task was to find the **maximum XOR** between the given value and any of the node values along the path from the root to the specified node. I initially proposed a brute force approach where, for each query, I would traverse the path from the root to the node and compute the XOR on the go. However, the interviewer asked me to optimize it. Recalling a previous problem I had solved involving maximum XOR using a **Trie**, I adapted that idea here. At first, I was constructing a new Trie for each query, but after a few hints from the interviewer, I realized that I needed to build the Trie dynamically **as I traversed the tree**, inserting values on the path and using a **counter mechanism** to keep track of the presence of values. While backtracking, I would decrement the counter or remove the value to reflect that the node was no longer in the current path. This allowed me to efficiently query for the maximum XOR at any point without rebuilding the Trie for every query.

After explaining the optimized approach using a Trie with dynamic insertion and backtracking, I was asked to code out the solution. I implemented the logic by building the Trie on-the-fly during a DFS traversal of the binary tree, maintaining a counter to manage insertions and removals as nodes entered and exited the current path. The interviewer seemed satisfied with both my thought process and implementation. The interview wrapped up with me asking him a couple of questions.

3. **[Hiring Managerial Round]** [14 candidates were shortlisted for round 2 (a mix of both HM and Tech Round 2]

After each panel finished, the students who showed strong technical skills were directly sent for the Hiring Manager (HM) round. I think this was done to check their communication and soft skills. In this round, many students were rejected even though they were very good at coding, so it was clear that this round tested more than just technical knowledge. My HM interview was taken by the Engineering Head at PhonePe. He started by asking how I was feeling and then asked me to introduce myself. He looked at my resume and began asking questions based on it. Since I had mentioned my internship, he started with that. I also told him about the freelance work I had done. He then asked in detail about the things I had built during my internship. After that, he moved on to my projects. All of my projects were from hackathons, so I was confident while explaining them. Then he asked me to choose one of my favorite apps and design its system. I explained my choices clearly and walked him through my thought process. At some points, he said my design might not handle large traffic, so I broke those parts down further and improved the design. In the end, he was happy with my final solution. The second half of the interview was mostly HR questions. I tried to answer honestly and gave real examples, even mentioning times when I was wrong and how my friends helped me understand things better. Showing a balanced and honest mindset helped. I ended the interview by asking him a few questions.

4. **[Technical Round 2]** [11 students were shortlisted for the final round]

This round was quite similar to the first technical round, where we were asked two DSA questions. The first question was: given a list of strings, return true if concatenating any two of them forms a palindrome. I started with the brute force approach, but the interviewer asked me to optimize it. I then explained a **Trie-based approach** where, before inserting a string into the Trie, I would reverse it and check if it already exists. I also maintained a pointer i to check if the remaining part of the string formed a palindrome. After some back and forth and dry runs, the interviewer was satisfied with my logic, asked about the time complexity, and then told me to write the code.

The second question was an array problem that was mostly **greedy/observational**. Despite several hints from the interviewer, I wasn't able to come up with the correct solution during the interview. He later shared the solution and asked me to **prove why the approach worked mathematically**. I tried to reason it out but couldn't come up with a complete proof. However, I did manage to explain a valid **DP-based approach** and correctly identified the DP state. Since time was up, we had to wrap the round, and I asked him a few questions before ending. I also requested him to walk me through the proof of the solution I couldn't solve. I checked the second question was rated **2700 on Codeforces** (Problem Link), which explained its difficulty level.

Out of the 11 candidates, 7 were selected in the end and I was one of them.

Useful Tips:

1. **Strong DSA skills are essential** – PhonePe places heavy emphasis on problem-solving, with this year's difficulty being higher than usual. Topics like Tries, Binary Lifting were commonly asked.
2. **CP is a bonus, not a requirement** – Being comfortable with CP is a big plus as it improves your problem solving skills. I didn't do any competitive programming, but regularly attempting Leetcode contests and upsolving problems (sometimes trying to understand the 4th question) helped me grasp advanced topics and perform in a time bounded environment.
3. **Internship experience adds value** – My interview started with questions about my internship. It's important to clearly explain what you worked on and the impact you had.
4. **Know your resume inside out** – Be fully prepared to talk about everything on your resume, including freelance work, projects, and technical contributions.
5. **Hackathon projects stand out** – Projects from hackathons show real-world problem-solving and help you pitch confidently in interviews.
6. **Good communication is crucial** – Stay vocal, explain your thought process clearly, and engage with the interviewer. Many technically strong candidates were rejected due to weak soft skills.
7. **Be consistent, not rushed** – I couldn't even clear Barclays OA for the summer internship, but consistent DSA practice with concept clarity (not rote learning) made a big difference. I would say even 6-8 months of sincere dedication toward DSA can help you clear any OA.

Resources:

1. **DSA**: Follow Striver's A2Z DSA Sheet – it's structured, beginner-friendly, and covers all essential topics and patterns.
2. **SQL**: Practice from LeetCode's Top 50 SQL Questions – great for query writing and real interview problems.
3. **DBMS**: Watch this DBMS video – it covers everything you need for interviews.
4. **Core Subjects (OS, CN, OOPs)**: Refer to **GeeksforGeeks** – it's sufficient for brushing up on the most asked concepts in interviews.
5. **System Design**: Go through System Design Article.
   Tip: You can skim through it and skip advanced topics like Kafka.
6. **Tech Cheatsheet**: View the Cheatsheet here, I got this from my senior — it includes useful resources across key topics. Feel free to use and share!

Here's my resume that I used for PhonePe Resume

If you have any specific questions, pls feel free to reach out on LinkedIn

All the best for your placements!